

WEEK 5 Practical

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
This message is shown once a day. To disable it please create the
/home/divya/.hushlogin file.
divya@Divyasree:~$ mkdir producer_consumer
mkdir: cannot create directory 'producer_consumer': File exists
divya@Divyasree:~$ cd producer_consumer
-bash: cd: producer_consumer: Not a directory
divya@Divyasree:~$ mkdir producer_consumer
mkdir: cannot create directory 'producer_consumer': File exists
divya@Divyasree:~$ ls -l producer_consumer
-rwxr-xr-x 1 divya divya 16408 Sep 21 05:22 producer_consumer
divya@Divyasree:~$ mkdir producer_consumer_project
divya@Divyasree:~$ cd producer_consumer_project
divya@Divyasree:~/producer_consumer_project$ pwd
/home/divya/producer_consumer_project
divya@Divyasree:~/producer_consumer_project$ nano producer_consumer.c
divya@Divyasree:~/producer_consumer_project$ ls -l
total 4
-rw-r--r-- 1 divya divya 2419 Sep 26 04:51 producer_consumer.c
divya@Divyasree:~/producer_consumer_project$ gcc producer_consumer.c -o producer_consumer
divya@Divyasree:~/producer_consumer_project$ ./producer_consumer
Child consumed : 10.00 MB

----- Communication Efficiency -----
Data transferred : 10.00 MB
Time taken       : 0.003336 seconds
Throughput      : 2997.53 MB/s
divya@Divyasree:~/producer_consumer_project$ |
```

Code:

```
GNU nano 7.2 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#include <string.h>

#define DATA_SIZE (10 * 1024 * 1024)
#define BUFFER_SIZE 4096

int main() {
    int pipefd[2];

    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    // Parent = Producer
    if (pid > 0) {
        close(pipefd[0]);
```

```

    char buffer[BUFFER_SIZE];
    memset(buffer, 'A', BUFFER_SIZE);

    struct timespec start, end;

    clock_gettime(CLOCK_MONOTONIC, &start);

    int total_written = 0;

    while (total_written < DATA_SIZE) {
        int remaining = DATA_SIZE - total_written;

        int bytes_to_write =
            (remaining < BUFFER_SIZE) ? remaining : BUFFER_SIZE;

        int bytes_written =
            write(pipefd[1], buffer, bytes_to_write);

        if (bytes_written == -1) {
            perror("write");
            close(pipefd[1]);
            return 1;
        }

        total_written += bytes_written;
    }

    close(pipefd[1]);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1e9;

    double throughput =
        (total_written / (1024.0 * 1024.0)) / time_taken;

    printf("\n---- Communication Efficiency ----\n");
    printf("Data transferred : %.2f MB\n",
        total_written / (1024.0 * 1024.0));

    printf("Time taken      : %.6f seconds\n",
        time_taken);

    printf("Throughput       : %.2f MB/s\n",
        throughput);
}

// Child = Consumer
else {
    close(pipefd[1]);

    char buffer[BUFFER_SIZE];
    int total_read = 0;

    while (1) {
        int bytes_read = read(pipefd[0], buffer, BUFFER_SIZE);

        int total_read = 0;

        while (1) {
            int bytes_read = read(pipefd[0], buffer, BUFFER_SIZE);

            if (bytes_read == 0) {
                break;
            }

            if (bytes_read == -1) {
                perror("read");
                close(pipefd[0]);
                return 1;
            }

            total_read += bytes_read;
        }

        close(pipefd[0]);

        printf("Child consumed : %.2f MB\n",
            total_read / (1024.0 * 1024.0));

        exit(0);
    }

    return 0;
}

```

Develop a program that executes the equivalent of the shell command: `ls -l grep` using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
-bash: /home/divya/producer_consumer_project: Is a directory
divya@Divyasree:~$ cd ~
divya@Divyasree:~$ mkdir ls_grep_project
divya@Divyasree:~$ cd ls_grep_project
divya@Divyasree:~/ls_grep_project$ pwd
/home/divya/ls_grep_project
divya@Divyasree:~/ls_grep_project$ nano ls_grep.c
divya@Divyasree:~/ls_grep_project$ ls
ls_grep.c
divya@Divyasree:~/ls_grep_project$ gcc ls_grep.c -o ls_grep
divya@Divyasree:~/ls_grep_project$ ./ls_grep
-rw-r--r-- 1 divya divya 1294 Sep 26 05:02 ls_grep.c

Command executed: ls -l | grep c
divya@Divyasree:~/ls_grep_project$ |
```

Code:

```
GNU nano 7.2 ls_grep.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    // Create pipe
    if (pipe(pipefd) == -1) {
        perror("pipe");
        return 1;
    }

    // Create first child for ls -l
    pid1 = fork();

    if (pid1 == -1) {
        perror("fork");
        return 1;
    }

    if (pid1 == 0) {
        // Child 1: execute ls -l

        close(pipefd[0]);

        // Redirect standard output to pipe
```

```

        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    // Create second child for grep
    pid2 = fork();

    if (pid2 == -1) {
        perror("fork");
        return 1;
    }

    if (pid2 == 0) {
        // Child 2: execute grep

        close(pipefd[1]);

        // Redirect standard input from pipe
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);

        execlp("grep", "grep", "c", NULL);

        perror("execlp grep");
    }

```

^G Help
 ^X Exit

^O Write Out
 ^R Read File

^W Where Is
 ^\ Replace

^K Cut
 ^U Paste

^T Execute
 ^J Justify

^C Location
 ^/ Go To Line

M-U Undo
 M-E Redo

```

        perror("execlp grep");
        exit(1);
    }

    // Parent process
    close(pipefd[0]);
    close(pipefd[1]);

    // Wait for both children
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("\nCommand executed: ls -l | grep c\n");

    return 0;
}

```